IDscanner

Codebase Reference Guide

Architecture · Flows · State · Threading

1. Big Picture — How All Files Connect

IDscanner is a PyQt6 desktop app that scans physical IDs (Passport, National ID, Driver's License) via camera or file upload. It runs OCR/QR detection in background threads, then shows results on a review screen where users can save output as text or XML.

The six source files and what they own:

File	Responsibility
main.py	App entry point. Owns all shared state (front_file, back_file, pendingResponse, lastResult, etc.). Routes button presses via go_next() / go_back().
load_ui.py	Loads the Qt Designer .ui file, wires every button/signal to its handler. Runs once at startup.
cam_handler.py	Controls the camera (MindVision SDK or OpenCV fallback). Feeds a 30ms timer loop. Writes captured frames to disk.
file_handler.py	Handles manual file upload buttons. Also owns the standalone convert_pdf_pages() helper used by pdf_preview_handler.
pdf_preview_handler.py	Handles the PDF upload flow end-to-end: renders preview, runs background detection + OCR thread, stores results, navigates to review.
review_handler.py	Populates the review page (images + text), formats results, and handles download (TXT or XML).
inference_handler.py	Calls the actual OCR/QR functions from inference.py, validates results, and formats them for display. (Not provided but referenced throughout.)
inference.py	Low-level image processing: detect_and_crop_id(), scan_passport(), scan_national_id(), scan_driver_license(), auto_detect_all_ids(), etc.

Dependency arrows (A → B means A imports B):

```
main.py
  ├── load_ui.py          (wires signals, then done)
  ├── cam_handler.py      (camera loop)
  ├── file_handler.py     (upload buttons)
  ├── pdf_preview_handler.py  (PDF flow)
  ├── review_handler.py   (review page)
  └── inference_handler.py    (OCR calls)
        └── inference.py  (image processing)
```

> **Key insight:** main.py is the single source of truth for all shared state. Every handler receives a reference to the MainWindow (called 'p' or 'parent') and reads/writes state on it directly.

---

2. Shared State on MainWindow

All inter-handler communication happens through attributes on the MainWindow instance. Here is every important attribute and what it means:

Attribute	Meaning / Set By / Read By

front_file	Dict {path, name, size, status, side}. The front image of NID or DL. Set by file_handler or pdf_preview_handler. Read by review_handler.
back_file	Same as front_file but for the back side.
uploaded_files	List of file-info dicts for the Passport upload flow (list widget). Managed by FileManager.
current_frame	The most recent raw BGR numpy array from the camera. Written every 30ms by CamHandler.update_frame().
captured_frame	Snapshot for Passport camera flow. Set by CamHandler.capture_image(). Deleted by reset_session().
captured_front_frame	Front snapshot for NID/DL camera flow. Set/deleted by CamHandler.toggle_capture().
captured_back_frame	Back snapshot for NID/DL camera flow. Set/deleted by CamHandler.toggle_capture().
pendingResponse	The raw OCR result dict from inference (not yet shown). Written by InferenceHandler callbacks. Consumed (and cleared) by ReviewHandler.show_review_page().
pendingDebugImage	Path to a debug bounding-box image. Optional. Consumed by show_review_page() if debug_mode is on.
lastResult	A permanent copy of pendingResponse saved before it is cleared. Used by download_text() for XML export.
lastIdType	String: 'Passport', 'National ID', or 'Driver\'s License'. Saved alongside lastResult.
debug_mode	Boolean. Toggled by the debug checkbox. When True, review page shows an extra bounding-box tab.
page_history	List of page indices. go_back() pops from here to know where to return.
pdf_preview	PdfPreviewHandler instance. Created after UiLoader so Qt widgets exist.
pdfDetectionStatus	QLabel on page 7 (PDF preview page). Updated by PdfPreviewHandler._set_status().

3. The Four Main User Flows

All flows start on page 0 (home). The user picks an ID type from the dropdown and selects an input method. go_next() reads those choices and routes accordingly.

3A. Passport — Camera

#	Where	What Happens
1	Home (page 0)	User selects 'Passport' + Camera, clicks Next. go_next() calls camera.start_camera(), navigates to page 1.
2	Page 1 (camera)	Camera loop runs. User clicks Capture → cam_handler.capture_image() stores captured_frame, saves JPEG to disk, fires InferenceHandler.infer_page2_camera_passport() in a QTimer callback.
3	Background	infer_page2_camera_passport() calls scan_passport(captured_frame). On completion it writes pendingResponse and enables the Continue button.

4	Page 1 → Page 3	User clicks Continue. go_next() calls inference.validate_passport_result_sync(). If valid, navigates to page 3 (review).
5	Page 3 (review)	camera.stop_camera() called. review_handler.show_review_page() runs: reads & clears pendingResponse → saves to lastResult / lastIdType, populates resultbox and image tabs.
6	Download	User clicks Download. review_handler.download_text() opens Save dialog. Uses lastResult + lastIdType for XML export.

3B. Passport — File Upload

#	Where	What Happens
1	Home (page 0)	User selects 'Passport' + Upload, clicks Next. go_next() navigates to page 2 and immediately calls files.upload_image(uploadedImageView, None).
2	Page 2	FileManager.upload_image() opens file picker. For images: copies to output folder, appends to uploaded_files, calls trigger_inference() which fires infer_page3_upload_passport().
3	Background	infer_page3_upload_passport() runs OCR. Writes pendingResponse, enables Continue.
4	Page 2 → Page 3	go_next() validates, navigates to review. Same review flow as 3A step 5–6.

3C. NID / DL — Camera

#	Where	What Happens
1	Home (page 0)	User selects NID or DL + Camera. go_next() navigates to page 4. Camera starts.
2	Page 4	Two capture buttons: captureButtonp2 (front) and captureButtonp3 (back). Each calls toggle_capture() which sets captured_front_frame or captured_back_frame.
3	Auto-trigger	When both frames exist, toggle_capture() calls infer_only_driver_license_camera() or infer_only_national_id_camera() in a QTimer.
4	Page 4 → Page 3	go_next() validates result. For DL calls validate_driver_license_result_sync(), for NID calls validate_national_id_result_sync(). Then navigates to review.
5	Review	Same as 3A step 5–6.

3D. NID / DL — PDF Upload (the complex one)

#	Where	What Happens
1	Home (page 0)	User selects PDF radio button. go_next() opens QFileDialog inline (not via FileManager), calls convert_pdf_pages() to get numpy arrays.
2	Page 6 (preview)	pdf_preview.load_pdf(pages, path) is called. It renders pages in the scroll area, populates fileListWidget_2, pushes page 0 onto page_history, sets currentIndex to 6.
3	Background thread	PdfPreviewHandler._worker_thread() starts immediately. It calls auto_detect_all_ids() to find the ID type, then QR-scans every page to identify front vs back, then runs the appropriate scan_*() function.

4	Signal back to UI	Worker emits scan_finished signal → _on_scan_finished() runs on main thread. Writes pendingResponse, updates idOption dropdown, sets status label to 'Scan complete'.
5	User clicks Continue	go_next() on page 6 calls pdf_preview.on_continue(). Checks detection/scan flags. Calls _proceed() which saves images via _save_images(), sets lastResult/lastIdType, navigates to page 3, calls show_review_page().

> **Why page_history is manually managed in load_pdf():** Normally go_next() appends to page_history before navigating. But load_pdf() is called before go_next() returns (inside the pdf_checked branch), so it manually pushes 0 onto page_history to ensure Back works correctly from the preview page.

---

4. Threading & The Signal/Slot Pattern

Why threading?

OCR and QR decoding are CPU-heavy operations that would freeze the UI if run on the main Qt thread. The solution: run them on a background daemon thread, then signal back to the main thread to update the UI.

The pattern used in PdfPreviewHandler

```
# 1. Define signals on a QObject subclass (must be class-level)
class WorkerSignals(QObject):
    scan_finished = pyqtSignal(object)   # carries the result payload
    status        = pyqtSignal(str)       # carries a status string

# 2. Connect signals to main-thread slots in __init__
self.signals = WorkerSignals()
self.signals.scan_finished.connect(self._on_scan_finished)  # runs on main thread
self.signals.status.connect(self._set_status)               # runs on main thread

# 3. Launch the worker
threading.Thread(target=self._worker_thread, daemon=True).start()

# 4. Inside worker, emit instead of touching Qt widgets directly
self.signals.status.emit('Scanning...')       # safe cross-thread call
self.signals.scan_finished.emit(results)      # triggers _on_scan_finished
```

> **Critical rule:** Never touch Qt widgets from a background thread. Always emit a signal and let the connected slot (which runs on the main thread) do the widget update.

QTimer.singleShot — the simpler pattern

For camera and file upload flows, background work is kicked off with QTimer.singleShot(100, callback). This schedules the callback to run 100ms later on the main thread's event loop — it does NOT create a background thread. This is fine because the inference functions in those paths are synchronous and tolerable in latency, but it still keeps the UI responsive during the brief delay.

```
# Pattern seen in file_handler.py and cam_handler.py:
p.continuep3.setEnabled(False)   # disable button immediately
QTimer.singleShot(100, p.inference.infer_page3_upload_passport)  # run after 100ms
# When infer_page3_upload_passport finishes it re-enables the button
```

daemon=True — what it means

The background thread in PdfPreviewHandler is created with daemon=True. This means the thread will be killed automatically when the main application exits, rather than keeping the process alive. Without this, closing the window while a scan is running could hang the process.

Race condition guard

on_continue() checks three flags before proceeding: _detection_done, _scan_done, and whether _scan_results is non-empty. This prevents the user from clicking Continue before the background thread has finished its work.

```
def on_continue(self):
    if not self._detection_done:
        QMessageBox.information(..., 'ID detection is still running...')
        return
    if self._scan_done and not self._scan_results:
        QMessageBox.warning(..., 'No IDs were detected...')
        return
    if not self._scan_done:
        QMessageBox.information(..., 'Scanning is still in progress...')
        return
    self._proceed()   # safe to proceed
```

5. Navigation — Pages, History & go_next()

Page index map

Index	Page / Screen
0	Home — ID type dropdown + input method radio buttons
1	Passport Camera — live camera view, capture button
2	Passport Upload — file list widget, upload button
3	Review — tab widget with images + result text box + download
4	NID/DL Camera — two camera views (front & back)
5	NID/DL Upload — front/back upload buttons
6	PDF Preview — scroll area with rendered pages, status label

page_flow dict

For pages that always go to the same destination, the mapping is stored in a dict so go_next() doesn't need if/else chains:

```
self.page_flow = {
    1: 3,   # Passport camera  → Review
    2: 3,   # Passport upload  → Review
    4: 3,   # NID/DL camera    → Review
    5: 3,   # NID/DL upload    → Review
    3: 0,   # Review           → Home (resets session)
}
# Page 0 and page 6 are NOT in page_flow — they need custom routing logic.
```

go_next() logic summary

go_next() always pushes current page onto page_history first, then:

- Page 6 (PDF preview): delegates to pdf_preview.on_continue(), pops history if it returns early.
- Pages in page_flow: validates preconditions (was image captured? do files exist? did OCR pass?). If a check fails it pops history and returns. Otherwise navigates to the mapped page.
- Page 0 (home): reads idOption + radio buttons to decide which page to go to. PDF path calls pdf_preview.load_pdf() and returns immediately without touching page_flow.

go_back()

Simply pops page_history and sets the page index. If going back to page 0 or from page 3, it calls reset_session() to clear all state.

reset_session()

Clears all captured frames, uploaded files, front/back file info, pending OCR response, and camera state. Does NOT reset idOption or the radio buttons — the user keeps their last selection.

---

6. Key Data Structures

file_info dict

```
# Created by make_file_info() in file_handler.py
{
  "path":   "/abs/path/to/image.jpg",
  "name":   "image.jpg",
  "size":   "1.24 MB",
  "status": "Completed",
  "side":   "front"  # or "back" or None (for passport upload)
}
```

pendingResponse / lastResult — shape by ID type

Passport

```
{
  "parsed": {
    "Passport/MRZ": {
      "Surname": "DOE",
      "Given_names": "JOHN",
      "Sex": "M",
      "Birth_date": "850101",
      "Nationality": "USA",
      "Document_number": "A12345678",
      "Country": "USA",
      "Expiry_date": "300101"
    }
  },
  "valid": True,
  "debug_image": "/path/to/debug.jpg"  # only if debug_mode=True
}
```

National ID

```
{
  "qr": {
    "NationalID/QR": {
      "subject": { "lName": ..., "fName": ..., "mName": ...,
                   "sex": ..., "DOB": ..., "POB": ..., "PCN": ... },
```

```
      "Issuer": ...,
      "DateIssued": ...
    },
    "valid": True
  },
  "front": {
    "parsed": { "NationalID/Front": { "Address": ... } }
  },
  "match": { "passed": True, ... },
  "valid": True
}
```

Driver's License

```
{
  "parsed": {
    "Driverslicense/OCR": {
      "Name": ..., "Sex": ..., "Birthdate": ...,
      "Address": ..., "License No": ..., "Expiration Date": ...
    }
  },
  "valid": True
}
```

---

7. The Tricky Parts — What to Watch Out For

pendingResponse is consumed once

ReviewHandler.show_review_page() reads pendingResponse, copies it to lastResult, then sets pendingResponse = None. This means if you call show_review_page() twice you'll get an empty resultbox the second time. The PDF flow avoids this by also writing lastResult/lastIdType in _proceed() before calling show_review_page().

PDF flow manually manages page_history

The PDF code path in go_next() is unusual: it calls pdf_preview.load_pdf() which itself calls p.page_history.append(0) and p.Form1.setCurrentIndex(6) — then returns without going through the normal page_flow routing. If you trace go_next() naively you'd think page_history gets double-pushed, but load_pdf() navigates away before page_flow code runs.

continuep7 vs continuep6 — naming confusion

Page 6 in the stacked widget is the PDF preview page, but the Continue button is named continuep7 in the Qt Designer file (the designer named pages starting from 1, the code indexes from 0). The mapping: continuep7 → page index 6. This is wired in load_ui.py.

MindVision SDK fallback

CamHandler tries to import mvsdk at module load time. If it fails (e.g. SDK not installed), MVSDK_AVAILABLE is set to False and all camera operations silently fall back to OpenCV. You can test without the industrial camera hardware this way.

QR-first back detection in pdf_preview_handler

The PDF worker does not assume page order. It scans every page looking for a QR code. The page with a QR code is the back (the QR side of NID/DL). The remaining page is treated as front. Only if no QR is found anywhere does it fall back to positional order (page N = front, page N+1 = back).

save_image copies a numpy frame to disk

```
# file_handler.py
def save_image(image: np.ndarray, folder: str, filename: str) -> str:
    os.makedirs(folder, exist_ok=True)
    path = os.path.join(folder, filename)
    cv2.imwrite(path, image)    # BGR numpy → JPEG/PNG on disk
    return path
# The returned path is stored in file_info dicts so review_handler
# can later read it back with cv2.imread(path) to display the image.
```

Why lastResult / lastIdType were missing originally

The download_text() function needs lastResult and lastIdType to build XML. In an earlier version of main.py these were never initialized in __init__, so if the user clicked Download before any review was shown, Python threw an AttributeError. The fix was simply to add self.lastResult = None and self.lastIdType = None in MainWindow.__init__() — which is why main.py's change log notes 'ADDED: was missing, caused download errors'.

---

8. Quick Reference — Where To Find Things

I want to...	Go to...
Add a new page/screen	main.py: add to page_flow dict or add a case in go_next(). Also add Back/Continue buttons to load_ui.py.
Change what happens when an ID is captured by camera	cam_handler.py: capture_image() or toggle_capture()
Change what happens when a file is uploaded	file_handler.py: finalise_upload() or trigger_inference()
Change the PDF background detection logic	pdf_preview_handler.py: _worker_thread()
Change how results are displayed as text	inference_handler.py: format_pending_response()
Change the XML export structure	review_handler.py: format_as_xml()
Add a new supported ID type	inference.py (add scan function), inference_handler.py (add validate + format), review_handler.py (add XML branch), main.py go_next() (add routing), idOption dropdown in .ui file
Debug a blank review page	Check pendingResponse isn't None before show_review_page() is called. Check lastResult for download issues.
Debug a frozen UI during scan	The scan is probably running on the main thread instead of a background thread. Use threading.Thread or QTimer.singleShot.